University of North Carolina at Charlotte
Department of Electrical and Computer Engineering

Homework Report 2

ECGR 5106 – Introduction to Deep Learning

Name: Viprav Lipare

Student #: 810288922

Date: June 21st, 2026

GitHub: https://github.com/vipravlipare/ECGR-5106-Intro-To-Deep-Learning

Homework 2 Report

Problem 1: RNN, LSTM, and GRU on the Homework Sequence

Objective

For this problem, character-level next-character prediction models were trained on the long sequence given in the homework. The goal was to compare a basic RNN, LSTM, and GRU using sequence lengths of 10, 20, and 30. The same sequence, split, seed, optimizer style, and accuracy calculation were used for all three models so the comparison would be fair.

Assumptions

The problem was treated as a character-level classification problem, where the input is a sequence of characters and the model predicts the next character. An 80/20 train-validation split with seed 42 was used. In the improved run, the model predicted the next character at every time step in the sequence, not only the last character. This made better use of the homework text. The all-step top-1 held-out validation accuracy is reported as the main accuracy, and last-step accuracy is also reported for comparison. No separate test set was created in the notebook, so final held-out validation accuracy is used as the test-style comparison metric for this homework.

Model Setup

The model used an embedding layer, one recurrent layer, dropout, and one fully connected output layer. This setup was chosen because the assignment is focused on comparing recurrent cell types, so the surrounding architecture was kept simple and consistent. The embedding layer was used instead of one-hot vectors so the model could learn a compact representation of each character. A hidden size of 128 was chosen as a middle point: large enough to learn the paragraph patterns, but still small enough to train on a laptop CPU. The only recurrent layer changed between experiments:

RNN: nn.RNN(hidden_size=128)

LSTM: nn.LSTM(hidden_size=128)

GRU: nn.GRU(hidden_size=128)

The output layer predicted one of the vocabulary characters. Cross entropy loss was used because each step is a multi-class character classification problem. Dropout was included to reduce overfitting on the small text dataset. For complexity, the rough recurrent operation count from the notebook was used:

complexity = gates * layers * sequence_length * (hidden_size^2 + hidden_size^2)

RNN’s use 1 gate, GRU’s use 3 gates, and LSTM’s use 4 gates.

Training Setup and Hyperparameters

The hardware used was a laptop CPU. The seed was 42. AdamW was used because it combines Adam's stable adaptive learning rate behavior with decoupled weight decay, which helps regularize small neural networks. The learning rate was set to 0.0025 because it trained faster than a very small learning rate without becoming unstable. Weight decay=0.001, dropout=0.35, and label smoothing=0.05 were used to reduce overfitting on the short text sequence. Gradient clipping at 1.0 was used because recurrent models can have unstable gradients. ReduceLROnPlateau with patience 35 was used so the learning rate could decrease when validation loss stopped improving. The maximum epoch count was 350, but early stopping was used so most models stopped much before 350 epochs.

Problem 1 Results

The best overall result came from the GRU with sequence length 30. It reached 92.08% final held-out validation accuracy and 92.10% best held-out validation accuracy, so the final and best checkpoint results were almost identical. The longer sequence lengths worked better because they gave the model more context. RNN was the smallest and fastest model, but LSTM and GRU gave better validation accuracy because their gated memory structures can keep useful information for longer. For the main comparison, final held-out validation accuracy is used first, then best-checkpoint accuracy, runtime, parameter count, and complexity are used as secondary tradeoffs.

Problem 1 result table

Final accuracy means the final held-out validation accuracy from the completed run. Best accuracy is the best validation checkpoint during training. Complexity is a rough recurrent operation estimate from the notebook, so larger values mean the model uses more compute per sequence, not necessarily exact wall-clock time.

Model	Seq	Final Acc	Best Acc	Val Loss	Time (s)	Params	Complexity
RNN	10	74.19 %	74.19%	1.1691	53.59	44.6k	0.33M
RNN	20	86.43 %	86.53 %	0.8484	95.72	44.6k	0.66M
RNN	30	90.32 %	90.42 %	0.7366	172.26	44.6k	0.98M
LSTM	10	76.83 %	76.92 %	1.0773	75.19	143.7k	1.31M
LSTM	20	87.74%	87.79 %	0.7880	143.06	143.7k	2.62M
LSTM	30	91.71 %	91.72 %	0.6851	238.00	143.7k	3.93M
GRU	10	77.04%	77.21 %	1.0654	77.20	110.6k	0.98M
GRU	20	88.22%	88.31%	0.7568	185.64	110.6k	1.97M
GRU	30	92.08 %	92.10 %	0.6654	204.06	110.6k	2.95M

Discussion

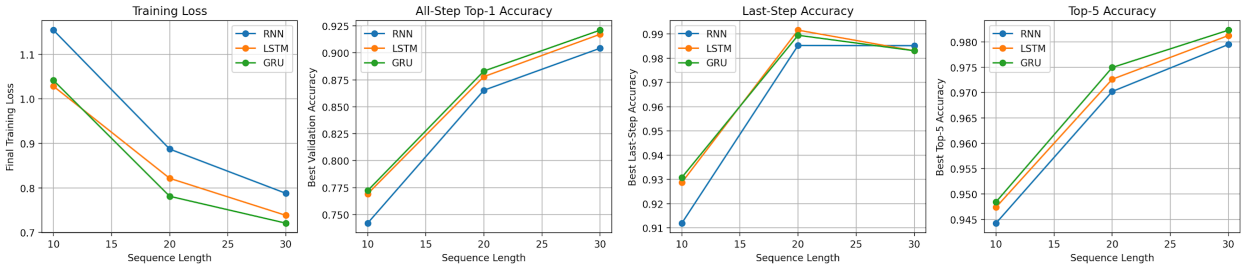
The sequence length had a major effect. Going from length 10 to 30 improved every model, and the improvement was large enough that sequence length should be considered one of the main factors in this problem. For example, GRU improved from 77.21% best validation accuracy at length 10 to 92.10% at length 30. LSTM improved from 76.92% to 91.72%, and the basic RNN improved from 74.19% to 90.42%. This shows that the models were using the extra context to make better predictions over the longer text sequence.

The loss curves support the same conclusion. The length 20 and length 30 runs had noticeably lower validation loss than length 10, and the training and validation losses stayed fairly close. That matters because it means the stronger validation accuracy was not only from memorizing the training text.

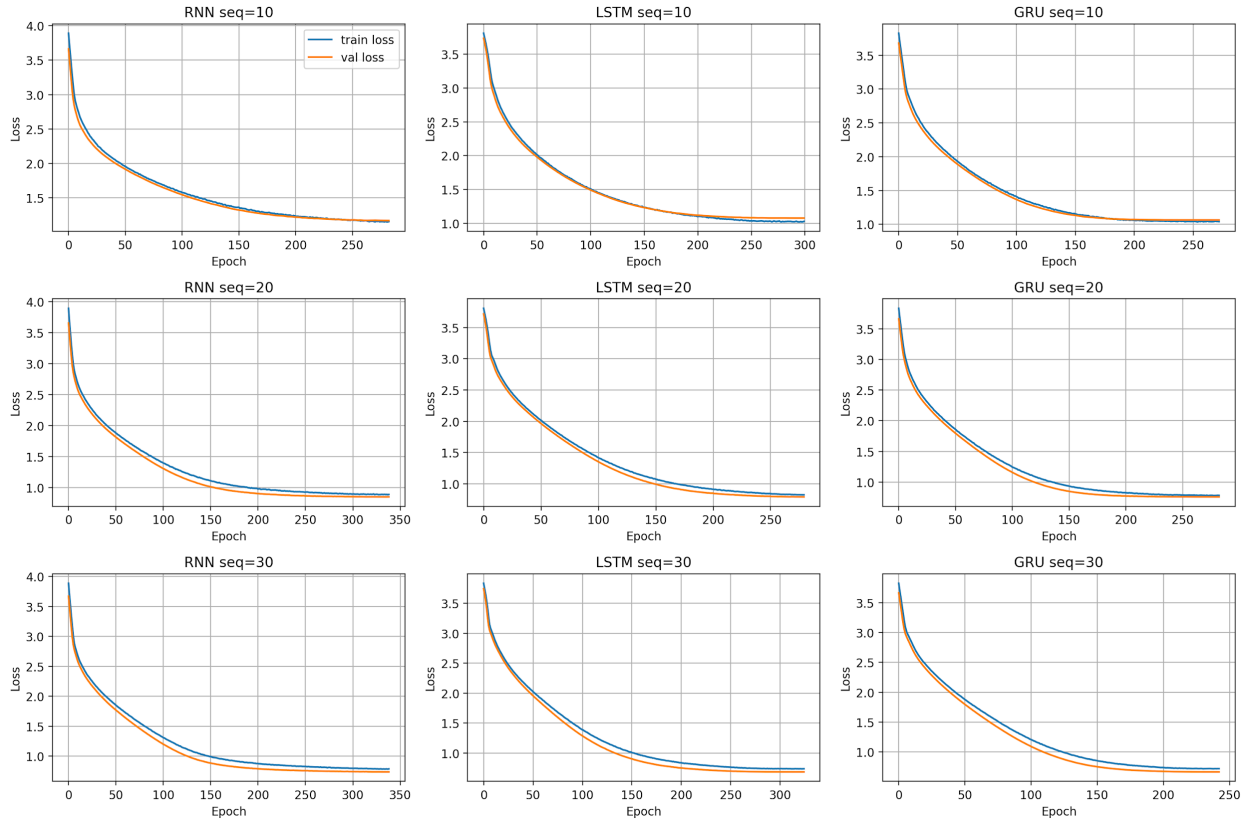
For the model comparison, the GRU was the best overall choice. The GRU length 30 model had the highest validation accuracy, the lowest validation loss, and only 110,637 parameters. The LSTM was close in accuracy, but it used 143,661 parameters because the LSTM has four internal gates. The simple RNN was fastest and smallest, but it consistently had the lowest accuracy because it had no gates to help control long-term memory.

Based on the results, the best Problem 1 model is GRU with sequence length 30. It gave the best final held-out validation accuracy while still being smaller than LSTM. The main tradeoff is that longer sequence length increases computation because the recurrent layer has to process more time steps, but it does not increase the number of trainable parameters.

Problem 1 Plots



Problem 1 summary plot showing all-step top-1 accuracy and last-step accuracy. The main trend is that all three models improved as sequence length increased. The jump from length 10 to length 30 was larger than the difference between model families at the same length, so context length had the strongest effect. This means the models benefited more from having additional previous characters than from simply switching between LSTM and GRU. The all-step top-1 validation accuracy is the main comparison metric because it evaluates every predicted character in the sequence, while the last-step accuracy is included as an extra next-character check.



Problem 1 training and validation loss curves for the RNN, LSTM, and GRU runs. The curves show that the longer-sequence models converged to lower loss, especially for sequence lengths 20 and 30. The validation loss did not separate badly from training loss, so the models generalized reasonably well for this fixed text sequence instead of only memorizing the training split. The curves also show diminishing returns: length 30 was best, but it cost more computation than length 20. GRU length 30 reached the best final held-out accuracy with fewer parameters than LSTM, which supports choosing it as the best accuracy/efficiency tradeoff.

Problem 2: Tiny Shakespeare LSTM and GRU

Objective

For Problem 2, LSTM and GRU character models were trained on the Tiny Shakespeare dataset. The required sequence lengths were 20 and 30, and sequence length 50 was also tested. Hidden size, number of recurrent layers, and fully connected size were changed to see how the hyperparameters affected accuracy, runtime, perplexity, generated text, and model size.

Dataset and Model Setup

A character-level Tiny Shakespeare loader similar to the provided `shakespeare-loader.py` code was used. The provided loader downloads Tiny Shakespeare from the Karpathy character-RNN dataset, creates a sorted character vocabulary, maps characters to integer IDs, builds input

sequences and next-character targets, and wraps them in a PyTorch `Dataset` and `DataLoader`. The notebook follows the same idea, but the custom `CharDataset` creates each sequence and target inside `__getitem__` instead of pre-building all sequence tensors at once. This keeps the code close to the provided loader while using less memory.

Since the experiment was run on CPU, `fast_run=True` was used and the text was limited to 300,000 characters. This kept the runtime reasonable while still using a much larger and more realistic text dataset than Problem 1. The train-validation split was 80/20, matching the style of the provided loader code. An 80/20 split was used instead of 90/10 because the dataset was already large, and a larger validation portion gave a more stable held-out accuracy estimate. The batch size was 128, the seed was 42, the optimizer was Adam with learning rate 0.003, and each run trained for 10 epochs. Adam was chosen because it is stable for recurrent networks and does not require as much manual learning-rate tuning. A batch size of 128 was used as a speed/memory compromise for CPU training.

The model used an embedding layer, either an LSTM or GRU recurrent layer, one ReLU fully connected layer, and a final output layer over the vocabulary. This architecture was chosen because it directly matches the homework focus on recurrent language models. The embedding layer turns character indexes into learned vectors, the recurrent layer learns sequential patterns, and the fully connected layers convert the hidden state into character probabilities. The base hidden size was 128 because it was large enough to learn useful text patterns but still small enough to run on a laptop CPU. The smaller hidden-size experiment tested whether the model could be compressed, and the two-layer experiment tested whether deeper recurrent structure improved performance. The table has multiple LSTM and GRU rows because each model family was retrained under different required conditions: sequence length 20, sequence length 30, smaller hidden size, two recurrent layers, and sequence length 50.

The main metric is final held-out top-1 validation accuracy because no separate test split was created in the notebook. Top-1 accuracy means the model's single most likely predicted character must match the true next character. Best-checkpoint validation accuracy is also reported to show the best epoch reached during training. Top-3 accuracy means the true next character is counted as correct if it appears anywhere in the model's three highest-probability predictions. Top-5 accuracy uses the same idea but allows the correct character to appear in the five highest-probability predictions. These top-k metrics are useful for Tiny Shakespeare because the vocabulary is larger and several next characters can be reasonable after a phrase, even if only one exact character is counted as correct.

Part 1: Sequence Length 20 and 30 Results

The LSTM and GRU results were close, and the ranking depends on which accuracy type is emphasized. Top-1 accuracy is the strictest metric because the model only gets credit when its first choice is exactly correct. By that metric, the GRU sequence length 20 model was slightly highest at 52.62% final held-out accuracy. Top-3 and top-5 accuracy are less strict because they check whether the true next character appears in the model's top few guesses. Those metrics

are useful for language modeling because multiple characters may be plausible after the same context.

Using best-checkpoint accuracy, validation loss, top-5 accuracy, and perplexity, the LSTM sequence length 20 model was stronger overall, with 52.77% best validation accuracy, 83.39% top-5 accuracy, and 4.78 perplexity. In realistic terms, GRU seq20 had the slightly better final exact-match score, but LSTM seq20 produced a better probability distribution because it had lower loss, lower perplexity, and stronger top-k accuracy. Since the difference in top-1 accuracy was very small, the result is not a huge win for either model. It mainly shows that both gated models learned useful Shakespeare patterns, while LSTM had a small advantage in confidence/ranking quality.

The sequence length 30 models did not improve over sequence length 20. This is different from Problem 1. Tiny Shakespeare is much more varied than the fixed homework paragraph, so longer context does not automatically help unless the model has enough training time and capacity to use it. The validation losses stayed close to the training losses, so the models were not badly overfitting; the harder issue was that exact next-character prediction is much more ambiguous on the Shakespeare text. The bigger dataset also limited the number of epochs that could realistically be run on a laptop CPU. Because only 10 epochs were used, the models had less time to fully benefit from the longer sequence lengths.

For convergence speed, several models reached their best score near the final epoch, especially the LSTM models. This suggests that the runs were still improving or close to improving when training stopped. With more training time, the LSTM models would likely benefit more than the smaller models, but the CPU runtime made longer experiments less practical.

Tiny Shakespeare base result table

Final accuracy is the final held-out top-1 validation accuracy, meaning only the single most likely predicted character is counted. Best accuracy is the best validation checkpoint. Top-5 accuracy counts the prediction correct if the true next character is in the model's five most likely choices. Perplexity is calculated from validation loss, and lower perplexity means the model is less uncertain. Complexity is the rough recurrent operation estimate from the notebook; higher values mean more compute cost, even if CPU runtime does not scale perfectly.

Model	Seq	Final Acc	Best Acc	Val Loss	Top-5 Acc	Perplexity	Time (s)	Params	Complexity
LSTM	20	52.46%	52.77%	1.5646	83.39%	4.78	193.37	164.5k	2.62M
LSTM	30	52.53%	52.53%	1.5802	83.27%	4.86	238.55	164.5k	3.93M
GRU	20	52.62%	52.62%	1.5879	82.64%	4.89	471.89	131.5k	1.97M
GRU	30	52.02%	52.19%	1.612	82.37	5.0	280.1	131.5k	2.95M

				0	%	1	2		

Hyperparameter Experiments

For the hyperparameter tests, a smaller hidden state, a two-layer network, and the required sequence length 50 were compared. The smaller hidden-state models trained faster and used much less memory, but accuracy dropped. For example, the small LSTM used only 45,438 parameters, but its best validation accuracy was 51.09%, compared with 52.77% for the base LSTM. This shows that the smaller model did not have enough capacity to model the Shakespeare text as well.

Adding a second recurrent layer increased model size and training time, but it did not improve the best result. The two-layer LSTM used 296,638 parameters and took 443.62 seconds, but its best validation accuracy was 52.45%, slightly below the one-layer LSTM. This means the deeper model was not worth the extra cost in this CPU-limited setup. A deeper model might do better with more epochs, but for this assignment the base one-layer model was the better choice.

Hyperparameter result table

These rows compare capacity, depth, and sequence length changes.

Setup	Model	Seq	Hidden /Layers	Final Acc	Best Acc	Top-5	Perplexity	Time (s)	Params
Small	LSTM	20	64 / 1	50.98 %	51.09 %	82.04 %	5.23	153.92	45.4k
Small	GRU	20	64 / 1	50.97 %	50.97 %	81.59 %	5.24	247.97	37.1k
Two-layer	LSTM	20	128 / 2	52.45 %	52.45 %	82.88 %	4.83	443.62	296.6k
Two-layer	GRU	20	128 / 2	51.56 %	51.56 %	82.37 %	5.04	537.52	230.6k
Seq50	LSTM	50	128 / 1	52.30 %	52.30 %	83.42 %	4.80	293.13	164.5k
Seq50	GRU	50	128 / 1	52.02 %	52.02 %	82.81 %	4.91	318.01	131.5k

Sequence Length 50

When sequence length was increased to 50, the model complexity increased a lot because recurrent computation scales with sequence length. The LSTM seq50 model had rough

complexity 6,553,600, compared with 2,621,440 for LSTM seq20. That is about 2.5 times more recurrent computation.

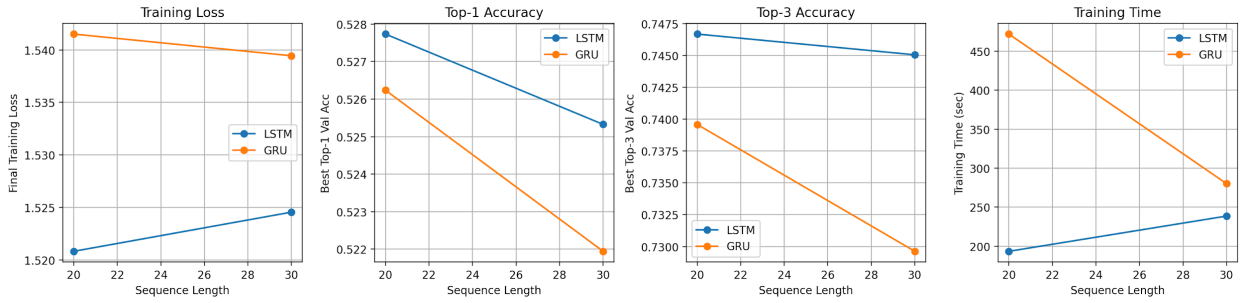
The longer sequence did not improve strict top-1 accuracy. The LSTM seq50 model reached 52.30%, which was slightly below the LSTM seq20 model at 52.77%. However, it had the best top-5 accuracy at 83.42%. This means the longer context helped the model keep the correct character near the top choices, but it did not help enough to make the exact first prediction better. Based on top-1 accuracy and runtime together, sequence length 50 was not the best practical choice.

Generated Text

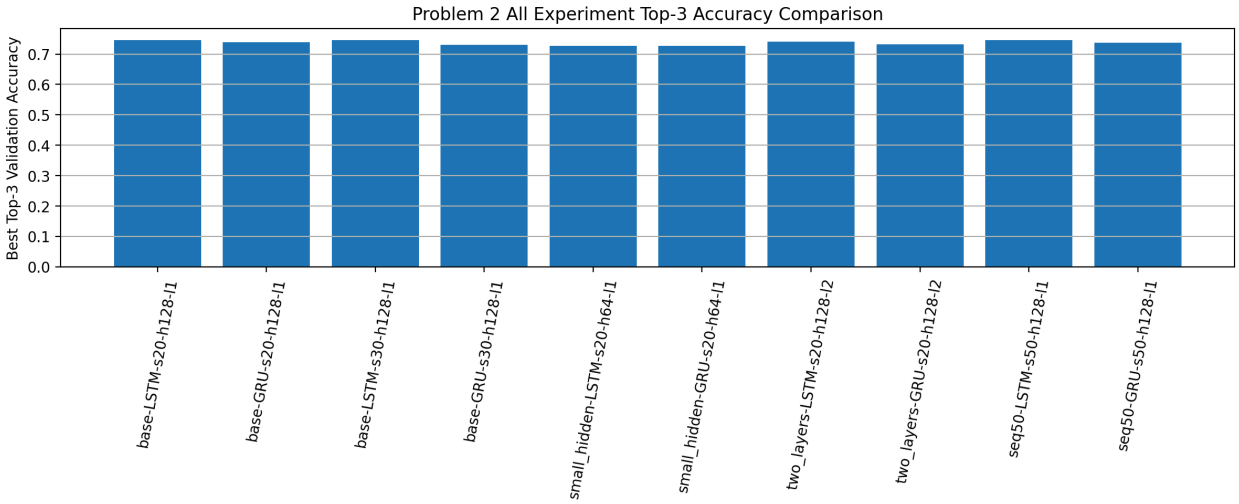
The generated text showed that the models learned common Shakespeare-style words, spacing, and short phrase patterns, but they also repeated phrases. For example, the LSTM seq20 model generated text like: To be, or not to be the come, the come, the come. The GRU seq20 model generated more varied text, but still repeated patterns such as: To be, or not to be sonest the see the senator.

This repetition is important to discuss because it shows the limitation of the model, not just the accuracy number. The models learned local character patterns, but greedy generation tends to choose the most likely next character repeatedly. This is why the generated text sounds partly structured but not fully human-like. Better sampling, more training time, or a larger model could improve the output sequence.

Problem 2 Plots



Problem 2 base LSTM/GRU comparison for sequence lengths 20 and 30. The figure shows that LSTM was slightly better than GRU on top-3 and top-5 accuracy, even though GRU used fewer parameters. It also shows that increasing from sequence length 20 to 30 did not improve the final held-out top-1 accuracy for this dataset. This trend is important because it shows that longer context alone was not enough for Tiny Shakespeare. The dataset is more varied than the fixed paragraph in Problem 1, so the model needs enough capacity and training time to actually use the extra context.



Problem 2 top-3 accuracy comparison across all saved model variants. The smaller hidden-size models are lower, which shows the effect of reduced capacity. The two-layer models are more expensive but not clearly better, which suggests that deeper architecture alone did not help under the 10-epoch CPU training limit. The sequence length 50 LSTM stayed strong in top-k accuracy, but its top-1 accuracy and runtime made it less practical than the base sequence length 20 models. Overall, the plot shows that adding capacity or context can help secondary metrics, but it does not guarantee a better final top-1 result.

Overall Comparison and Conclusion

For Problem 1, the best model was GRU with sequence length 30. It gave the highest final held-out validation accuracy while having fewer parameters than LSTM. RNN was fastest and smallest, but it gave the lowest accuracy. For Problem 2, Tiny Shakespeare was much harder. Using final held-out top-1 accuracy alone, the best base result was GRU sequence length 20 at 52.62%. Using best-checkpoint accuracy, top-5 accuracy, validation loss, and perplexity together, LSTM sequence length 20 was the strongest overall language model.

The main difference between the two problems is dataset difficulty. Problem 1 used one fixed paragraph, so the models could learn the repeated local patterns very well. Tiny Shakespeare had many speakers, punctuation styles, line breaks, and possible next characters, so exact top-1 prediction was much harder. This is why top-3 accuracy, top-5 accuracy, perplexity, and generated text were useful extra metrics for Problem 2.

The main trend is that sequence length helps a lot on the short homework sequence, but it does not automatically fix Tiny Shakespeare. On Tiny Shakespeare, the model needs more training time or better tuning to improve exact top-1 accuracy. Increasing model size also increased runtime and model size, so the best practical overall language model in this CPU setup was the one-layer LSTM with hidden size 128 and sequence length 20. If only final held-out top-1 accuracy is used, the GRU sequence length 20 model is the strict winner by a small margin.

Overall, the best model should be chosen using the required main metric first, then runtime and complexity second. For Problem 1 that means GRU sequence length 30. For Problem 2, final held-out top-1 accuracy points to GRU sequence length 20, while the broader language-model metrics point to LSTM sequence length 20.

Final best-result summary

Experiment	Best Model	Best Main Metric	Notes
Problem 1	GRU, seq length 30	92.08% final held-out validation accuracy	Highest final accuracy and fewer parameters than LSTM.
Problem 2 final top-1	GRU, seq length 20	52.62% final held-out validation accuracy	Slightly highest final top-1 result among the base models.
Problem 2 overall language model	LSTM, seq length 20	52.77% best validation accuracy, 83.39% top-5, 4.78 perplexity	Best overall balance of validation loss, perplexity, and top-k accuracy.
Problem 2 seq50 top-5	LSTM, seq length 50	83.42% top-5 validation accuracy	Best top-5 result, but higher sequence cost and lower final top-1 than GRU seq20.
Problem 2 small model	LSTM, hidden size 64	50.98% final held-out validation accuracy	Much smaller and faster, but slightly lower accuracy.

All source code and result files are included in the Homework_2 folder of my GitHub repository.